

## Feladat – „Húzzunk lapot 21-ig” konzolos játék

Készíts egy **C# konzolos alkalmazást**, amely egy egyszerű lap húzós játékot valósít meg.

### A játék menete:

---

- A játékos egymás után lapokat húzhat.
- Minden lap értéke **1 és 10 közötti véletlen szám**.
- A húzott lapok értéke **összeadódik**.
- A játékos minden körben eldönti, hogy:
  - húz még egy lapot (1),
  - vagy megáll (0).
- A játék addig tart, amíg:
  - a játékos meg nem áll, **vagy**
  - az összeg **21 fölé nem megy**.

### Játék kimenetek

---

A program a végén írja ki:

- **Pont 21 → győzelem**
- **21 alatt megáll → biztonságos megállás**
- **21 fölé megy → vereség**

### Kötelező metódusok

---

A programot **több metódusra bontva** kell megírni.

#### Main

**Feladat:** a program belépési pontja, elindítja a játékot és kiírja az eredményt.

#### General

**Feladat:** generál egy véletlen számot 1 és 10 között (lap húzása).

**Paraméter:** nincs

**Visszatérés:** egész szám

#### Tipp

**Feladat:** bekéri a játékos döntését, hogy húz-e még lapot.

- Csak **0 vagy 1** lehet érvényes bemenet.
- Hibás bemenet esetén újra kérdez.

**Paraméter:** nincs

**Visszatérés:** egész szám (0 vagy 1)

## **Jatek**

**Feladat:** a teljes játéklogika megvalósítása.

- Körök kezelése
- Lapok összegzése
- Játék leállási feltételek kezelése
- Játék eredményének meghatározása

**Visszatérési érték jelentése:**

- **2 - Pont 21 → győzelem**
- **1 - 21 alatt megáll → biztonságos megállás**
- **0 - 21 fölé megy → vereség**

**Paraméter:** nincs

**Visszatérés:** egész szám

## **Kiir**

**Feladat:** a játék eredményének kiírása a képernyőre.

- Győzelem
- Megállás
- Vereség

**Paraméter:** egész szám (a játék eredménye)

**Visszatérés:** nincs (void)

**Extra:**

- Írd ki a körök számát
- Legyen „Új játék?” kérdés
- Hibakezelés szöveges inputra
- AI ellenfél (aki 17-nél megáll)
- Statisztika: hány győzelem / vereség

## Megoldás

```
static void Main(string[] args) {
    Kiir(Jatek());
}

static int General() {
    return new Random().Next(1, 11);
}

static int Tipp() {
    int tipp;
    do {
        Console.WriteLine("Húzzunk egy lapot (1 - igen, 0 - nem) ?");
        tipp = Convert.ToInt32(Console.ReadLine());
    } while (tipp < 0 || tipp > 1);
    return tipp;
}

static int Jatek() {
    int lap = 0, huzott, tipp, n = 1;
    do {
        Console.WriteLine($"----- {n}. KÖR -----");
        tipp = Tipp();

        if (tipp == 1) {
            huzott = General();
            Console.WriteLine($"A húzott lap értéke: {huzott}");
            lap += huzott;
            Console.WriteLine($"Összesen a lapok értéke: {lap}");
            n++;
        }
    } while (tipp == 1 && lap <= 20);

    if (lap == 21) return 2;
    else if (lap < 21) return 1;
    else return 0;
}

static void Kiir(int eredmény) {
    if (eredmény == 2) Console.WriteLine("Gratulálunk, győztél!");
    else if (eredmény == 1) Console.WriteLine("Megálltál 21 előtt.");
    else Console.WriteLine("Sorry, ezt buktad.");
}
```

## Feladat – Nevek feldolgozása tömbben

Készíts egy **C# konzolos alkalmazást**, amely keresztneveket kér be a felhasználótól, majd különböző műveleteket végez a neveken.

### A program működése:

---

1. Bekér **N darab keresztnévet** (string).
2. Eltárolja őket egy tömbben.
3. Meghatározza:
  - o a nevek összes karakterének számát,
  - o a leghosszabb nevet,
  - o minden név **monogramját** (első + utolsó betű, nagybetűvel).
4. Kiírja az eredményeket.

### Kötelező metódusok

---

#### Main

##### Feladat:

- Feltölti a neveket tartalmazó tömböt
- Meghívja a feldolgozó metódusokat
- Kiírja az eredményeket

**Paraméter:** string tömb

**Visszatérés:** nincs (void)

#### Nevek

**Feladat:** Bekér egy keresztnévet a felhasználótól.

**Paraméter:** nincs

**Visszatérés:** string (a beírt név)

#### Feltölt

**Feladat:** Létrehoz egy adott méretű string tömböt, majd feltölti nevekkal a Nevek metódus segítségével.

**Paraméter:** egész szám (a nevek darabszáma)

**Visszatérés:** string tömb

#### HanyKarakter

**Feladat:** Összeszámolja, hogy a tömbben lévő nevek **összesen hány karakterből állnak**.

**Paraméter:** string tömb

**Visszatérés:** egész szám

## **LeghosszabbNev**

**Feladat:** Megkeresi és visszaadja a tömbben szereplő **leghosszabb** nevet.

**Paraméter:** string tömb

**Visszatérés:** string

## **ElsóUtolso**

**Feladat:** Minden névhez elkészíti a **monogramot**:

- első betű + utolsó betű
- nagybetűs formában

Az eredményt egy új string tömbben adja vissza.

**Paraméter:** string tömb

**Visszatérés:** string tömb

## Megoldás

```
static void Main(string[] args) {
    string[] nevek = Feltolt(5);
    Console.WriteLine("A nevek tömb ennyi karakter: " + HanyKarakter(nevek));
    Console.WriteLine("A leghosszabb név a listában: " + LeghosszabbNev(nevek));
    Console.WriteLine("Az egyes nevek monogramjai: " +
        string.Join(", ", ElsoUtolso(nevek)));
}

static string Nevek() {
    Console.WriteLine("Kérek egy keresztnévet:");
    return Console.ReadLine();
}

static string[] Feltolt(int db) {
    string[] nevek = new string[db];
    for (int i = 0; i < nevek.Length; i++) {
        nevek[i] = Nevek();
    }
    return nevek;
}

static int HanyKarakter(string[] nevek) {
    int db = 0;
    foreach (string nev in nevek) {
        db += nev.Length;
    }
    return db;
}

static string LeghosszabbNev(string[] nevek) {
    string max = nevek[0];
    foreach (string nev in nevek) {
        if (nev.Length > max.Length) max = nev;
    }
    return max;
}

static string[] ElsoUtolso(string[] nevek) {
    string[] rovid = new string[nevek.Length];
    int i = 0; char[] n;
    foreach (string nev in nevek) {
        n = nev.ToCharArray();
        rovid[i] = new string(new char[] { n[0], n[n.Length - 1]}).ToUpper();
        i++;
    }
    return rovid;
}

static string[] ElsoUtolso(string[] nevek) {
    string[] rovid = new string[nevek.Length];
    for (int i = 0; i < nevek.Length; i++) {
        string nev = nevek[i];
        rovid[i] = $"{nev[0]}{nev[^1]}".ToUpper();
    }
    return rovid;
}
```